

Exercise: BLE RSSI Trilateration in Python

BLE RSSI

We specifically want to simulate noisy RSSI measurements in a 5×10 m room, invert them to distances, and find the position by optimisation.

Parameters follow Bembenik & Falcman (2020): $\gamma = 1.74$, $f = 2.45$ GHz, TxPower $A = -10\gamma \log_{10}(4\pi f/c) \approx -35$ dBm, $\sigma_S = 3$ dBm RSSI noise.

The idea behind BLE positioning is straightforward. Small BLE beacons are fixed around the room at positions that are known in advance — measured once at setup and stored.

They just continuously broadcast a radio signal; they do not know where the receiver is. The receiver (e.g. a smartphone) listens to all beacons it can hear. Because radio signals weaken with distance — the further the beacon, the weaker the signal that arrives — the receiver uses the measured signal strength from each beacon as a proxy for distance

BLE RSSI Distance Model

When a radio signal travels distance ρ through space it loses power according to the **free-space path loss** model. In free space the signal spreads out in all directions — the further it travels, the more the received power drops. This is captured by:

$$\ell = \left(\frac{4\pi\rho}{\lambda} \right)^\gamma = \left(\frac{4\pi\rho f}{c} \right)^\gamma$$

The exponent γ controls how fast the signal decays with distance.

- In free space $\gamma = 2$, meaning power falls as $1/\rho^2$.
- In practice γ must be calibrated for the environment; we use $\gamma = 1.74$ taken from Bembenik & Falcman (2020), who measured it experimentally in an office room.

Interestingly this is below the free-space value of 2 — one might expect walls to absorb signal and cause faster decay, but reflections can constructively add energy toward the receiver, slowing it instead.

Taking the path loss into the log domain (converting to dBm) gives the RSSI formula, here added with noise:

$$S = \underbrace{-10\gamma \log_{10} \rho}_{\text{decreases as } \rho \text{ grows}} + \underbrace{10\gamma \log_{10} \left(\frac{4\pi f}{c} \right)}_{A = \text{constant at 1 m}} + \nu_i \quad \text{Where } \nu_i \sim \mathcal{N}(0, \sigma_S^2)$$

The first term is what carries the distance information — as ρ increases, S becomes more negative (weaker signal).

A is a constant that depends only on frequency and γ ; it is calibrated at 1 m by the manufacturer and stored in the beacon.

The noise $\nu_i \sim \mathcal{N}(0, \sigma_S^2)$ lumps together two sources of error. First, small electronic and quantisation noise from the **sensor** itself. More significantly, **multipath**: the signal bounces off walls and furniture so multiple copies arrive simultaneously; depending on their phase they add to or subtract from the direct signal, shifting the RSSI reading even when the receiver is perfectly stationary.

Signal inversion

Rearranging this, result in us being able to get distance from a measured RSSI reading:

$$\hat{\rho}_i = 10^{\frac{A-S_i}{10\gamma}}$$

Symbol Name		Meaning
ℓ	Path loss	Ratio of transmitted power to received power (dimensionless, > 1). Larger ℓ means more signal lost.
S	RSSI [dBm]	Received signal strength; more negative = weaker = further away
ρ	Range [m]	Distance from beacon to receiver
γ	Path-loss exponent	≈ 2 free space; 1.6–1.8 indoors (paper measured 1.736)
f	Frequency [Hz]	BLE: ≈ 2.4 GHz
λ	Wavelength	$\lambda = c/f$; for BLE at 2.4 GHz, $\lambda \approx 12.5$ cm.
A	TxPower [dBm]	RSSI at 1 m; calibrated by manufacturer and stored in beacon
c	3×10^8 m/s.	Speed of light, electromagnetic waves travel at this speed

Why not just time-of-flight?

Bluetooth — TOF impossible on standard hardware. Radio travels at $c = 3 \times 10^8$ m/s, so a 10 m gap takes only 33 ns to cross. Standard BLE chips have processing delays of ~ 150 μ s before software even registers a packet arrived — that delay alone gives a fictitious range of:

$$\frac{150 \times 10^{-6} \text{ s} \times 3 \times 10^8 \text{ m/s}}{2} = 22,500 \text{ m}$$

The travel time is completely buried in the processing overhead, so RSSI is used instead — it costs nothing extra since every BLE chip measures received power as part of normal operation.

Games-on-Track and GPS — TOF works because of dedicated hardware and is more accurate. Systems like Games-on-Track and GPS use chips built specifically with nanosecond-precision timestamping circuits, making TOF both pos-

sible and far more accurate than RSSI.

Rather than measuring signal strength, both the transmitter and receiver generate the **same known pattern** — a chirp (Games-on-Track) or a pseudo-random code (GPS).

The receiver then computes the **cross-correlation** between its local copy and the incoming signal; the correlation peaks sharply at exactly the time shift where the two align, giving the travel time Δt and thus the range $r = c \Delta t$.

This is far more robust indoors than RSSI because it locks onto the **first-arriving** copy of the signal — the direct path — and ignores later reflections off walls entirely.

Optimisation-based trilateration

Because RSSI noise means the four estimated distances rarely place the receiver at exactly one consistent point, we instead find the position (x, y) that is most consistent with all four measurements simultaneously by minimising the sum of squared residuals:

$$\min_{x,y} \sum_{i=1}^4 \left[\underbrace{\sqrt{(x - b_{x,i})^2 + (y - b_{y,i})^2}}_{\text{geometric dist to beacon } i} - \underbrace{\hat{\rho}_i}_{\text{estimated dist}} \right]^2$$

For each beacon i the bracket computes the geometric distance from the candidate position (x, y) to the known beacon location $(b_{x,i}, b_{y,i})$, minus the distance estimated from the RSSI reading $\hat{\rho}_i$.

If (x, y) were placed exactly on beacon i 's circle the residual for that beacon would be zero — but that point very likely lie far from the other three circles, making their residuals large.

Summing over all four beacons prevents this: the solver is forced to find the position that is simultaneously close to all circles at once, which corresponds to the true receiver location. This is solved numerically with `scipy.optimize.minimize` (Nelder-Mead).

Results

With $\sigma_S = 3\text{ dBm}$ and 40 trials per location the simulation gives:

True position	Mean error	90th pct
(1.25, 2.50) m	2.95 m	5.07 m
(2.50, 5.00) m	2.27 m	4.46 m
(3.75, 7.50) m	2.46 m	4.39 m

Bembenik & Falcman report a raw baseline (no filtering) of 4.62 m mean and 6.64 m at the 90th percentile across two large irregular office rooms spanning roughly 20–30 m with walls between them. Our errors are lower since the 5×10 m room with four ideal corner beacons is a considerably simpler geometry — no irregular walls, no signal bleed from adjacent rooms, and beacons optimally placed at corners.

What the red dot clouds tell us. Each red dot is one complete trilateration result — the same position estimated from one noisy set of four RSSI readings. The spread of the cloud is the *positioning noise floor*: even standing perfectly still, each trial lands somewhere different because RSSI fluctuates due to multipath and sensor noise.

Unlike LiDAR where noise enters directly as small errors in range (~ 5 cm) and angle ($\sim 1^\circ$), here the noise is on the signal strength itself. As the beacon gets further away the signal weakens, so the same 3 dBm noise represents an increasingly large fraction of the total reading — the signal-to-noise ratio degrades with distance. The log scale makes this worse: the RSSI curve flattens at large distances, meaning a small change in S maps to a large jump in the estimated $\rho = 10^{(A-S)/(10\gamma)}$, translating 3 dBm of noise into **metres** of range uncertainty.

Why the estimates scatter so much. The signal from each beacon does not arrive via just one path. It bounces off walls, floor, and furniture, so **multiple copies** arrive simultaneously. RSSI measures the *total* received power — the direct copy plus all reflections combined. This blended power fluctuates moment to moment (a door opens, someone walks past) as reflections shift in and out of phase. Fluctuating RSSI gives a fluctuating range estimate, hence a different trilateration result each time. Typical indoor accuracy: **1–5 m**.